Lab Report: Datapath Storage Components

Course: CSC 364 - Computer Architecture

Objective

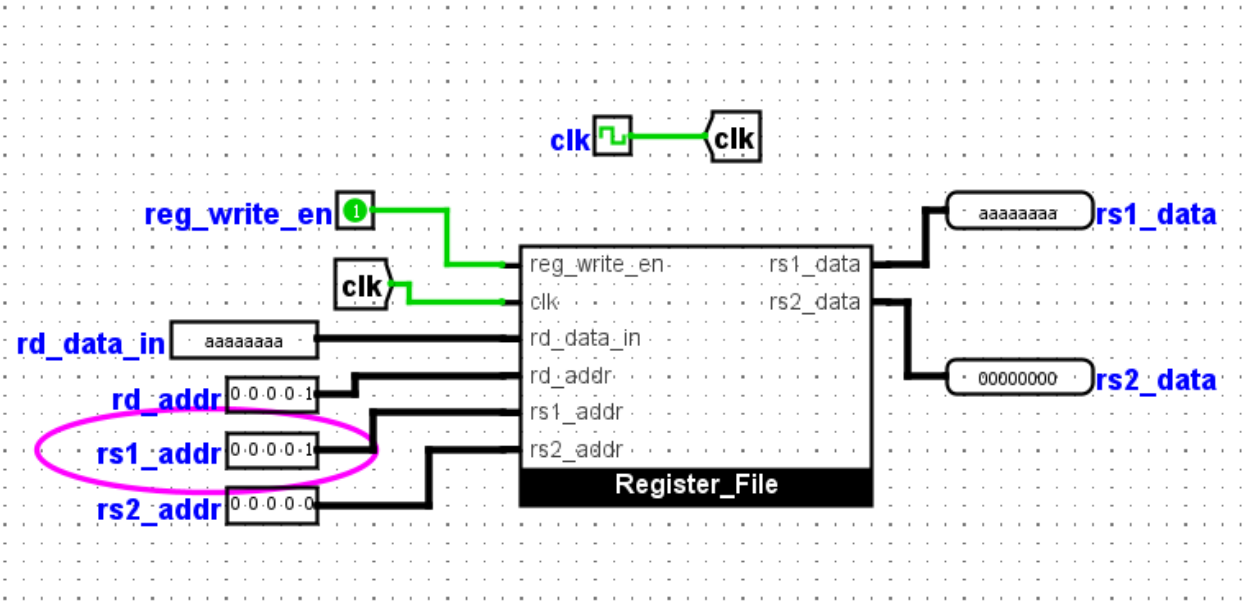
The goal of this lab was to design and implement two core storage components for a RISC-V processor: a 32 × 32-bit Integer Register File and a 4 GB byte-addressable Data Memory.

1. **32 x 32-bit Integer Register File**
   The Register File contains 32 registers, each 32 bits wide. It supports two combinational read ports (rs1_data, rs2_data) and one synchronous write port (rd_data_in).

   Key Design Features
   - **x0 Hardwiring:** Register 0 is hard-coded to zero. Any attempts to write to it are ignored, and it always outputs 0x00000000.
   - **Synchronous Writing:** Data is only saved to the selected destination register on the rising edge of the clock when reg_write_en is active.
   - **Combinational Reading:** Outputs update immediately when the address changes, without waiting for a clock pulse.

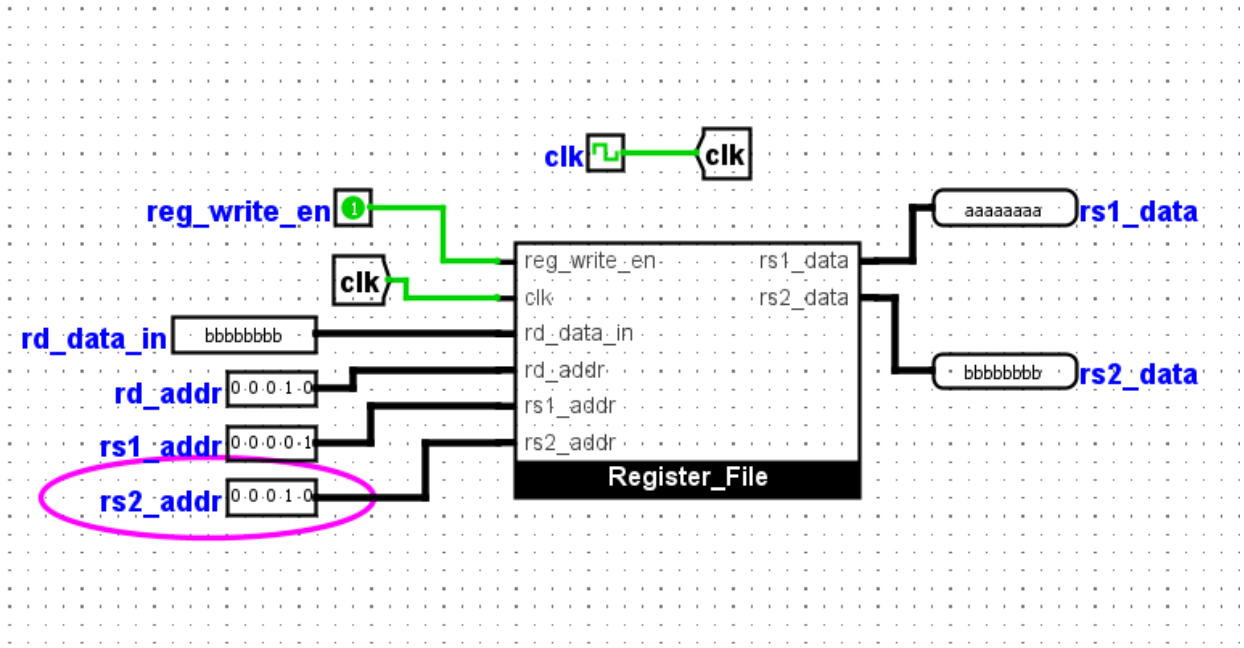
   Test Case 1: Synchronous Write to x1

   I set rd_addr to 1, rd_data_in to 0xAAAAAAAA, and enabled writing. After poking the clock, I set rs1_addr to 1 to verify the data was stored.



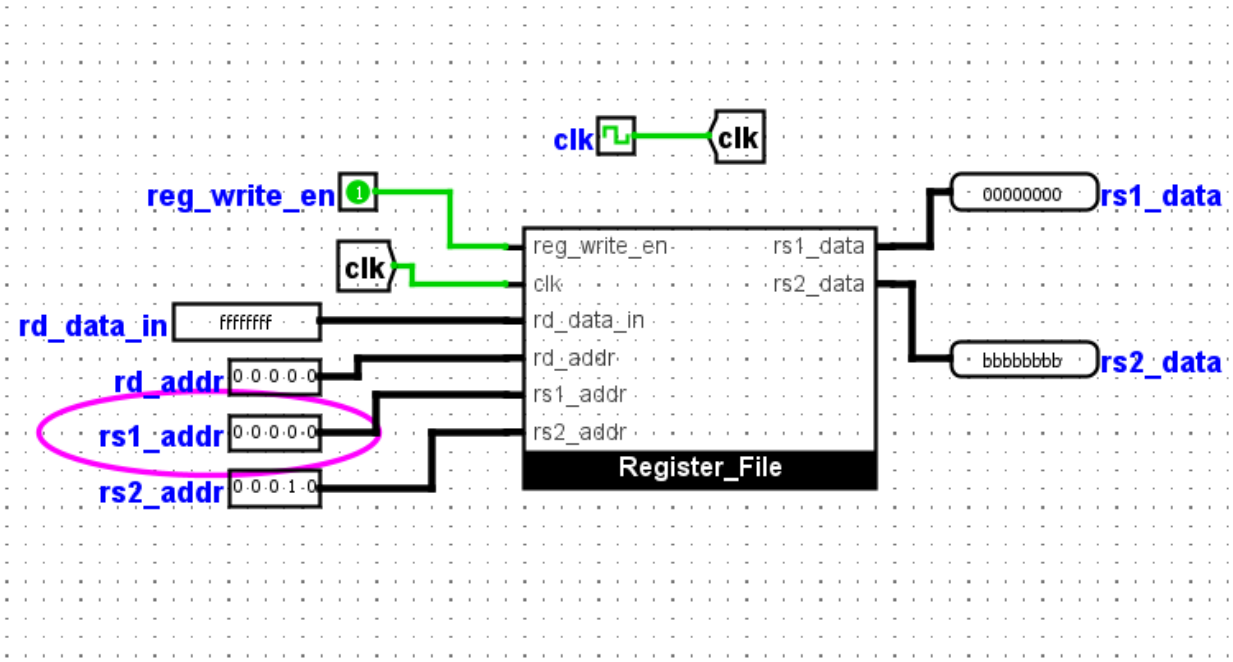
   Test Case 2: Dual Port Read (x1 and x2)

I stored 0xBBBBBBBB in register x2. I then set rs1_addr to 1 and rs2_addr to 2 to show that both values can be read at the same time.



Test Case 3: Register x0 Behavior

I attempted to write 0xFFFFFFFF to register 0. Upon reading it back, the output remained 0x00000000, proving that writes to x0 are correctly ignored.



2. Byte-Addressable Data Memory

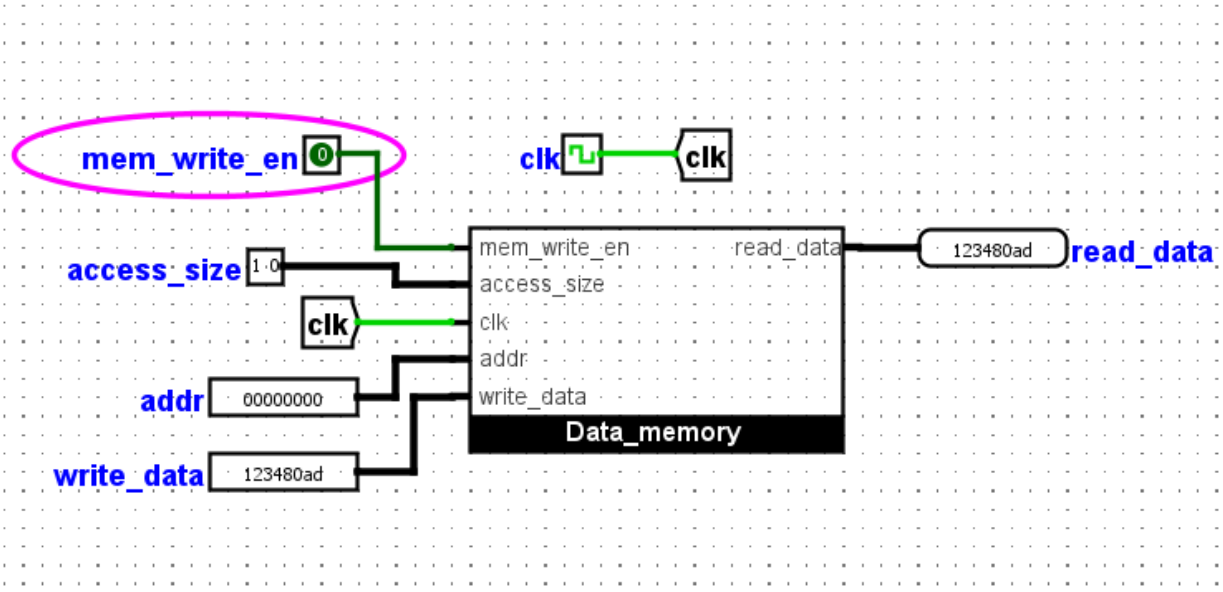
The Data Memory uses a 4-bank system to allow byte-level access. It supports three access sizes: Byte (8-bit), Halfword (16-bit), and Word (32-bit).

Key Design Features

- **4-Bank System:** Memory is split into four RAM banks (Bank 0 to Bank 3), each representing one byte of a 32-bit word.
- **Sign Extension:** When loading a Byte or Halfword, the circuit uses bit extenders to fill the upper bits. This ensures the 32-bit output correctly represents signed numbers.
- **Selective Writing:** Only the specific bank(s) targeted by the address and access size are updated during a write.

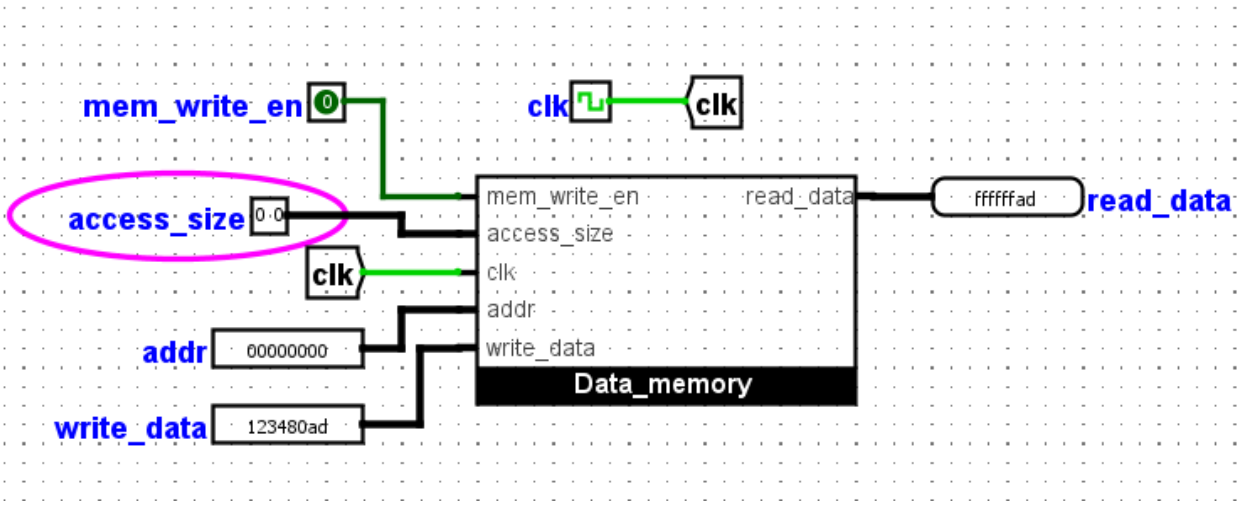
Test Case 4: Word Store and Load

I used the pattern 0x123480AD to test memory. I stored this as a full word at address 0.



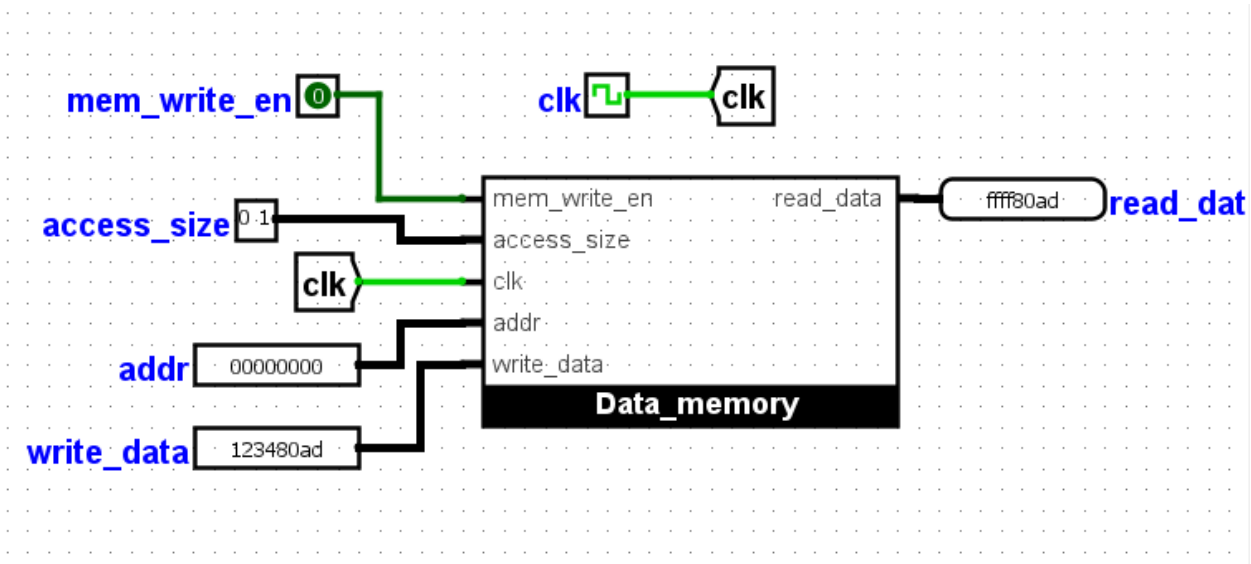
Test Case 5: Load Byte (Sign Extension)

I set the access size to Byte at address 0. The circuit correctly extracted 0xAD and sign-extended it to 0xFFFFFFAD because the highest bit was 1.



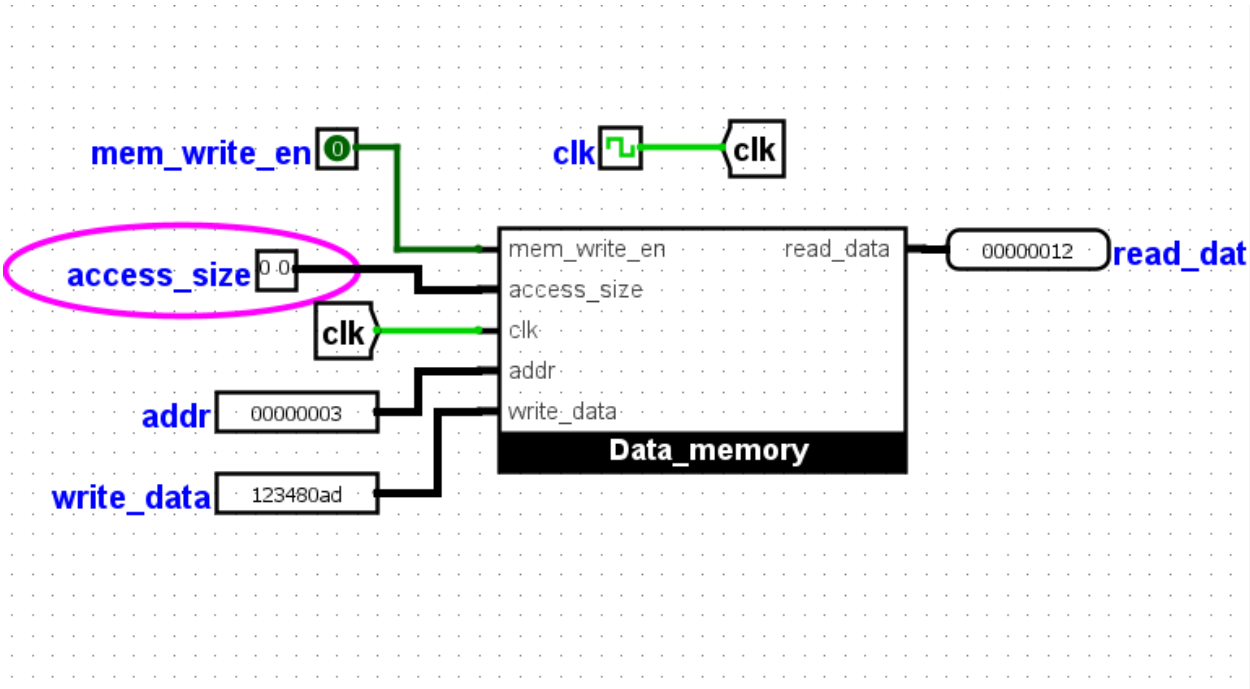
Test Case 6: Load Halfword (Sign Extension)

I set the access size to Halfword at address 0. The circuit combined Bank 0 and Bank 1 to get 0x80AD and sign-extended it to 0xFFFF80AD.



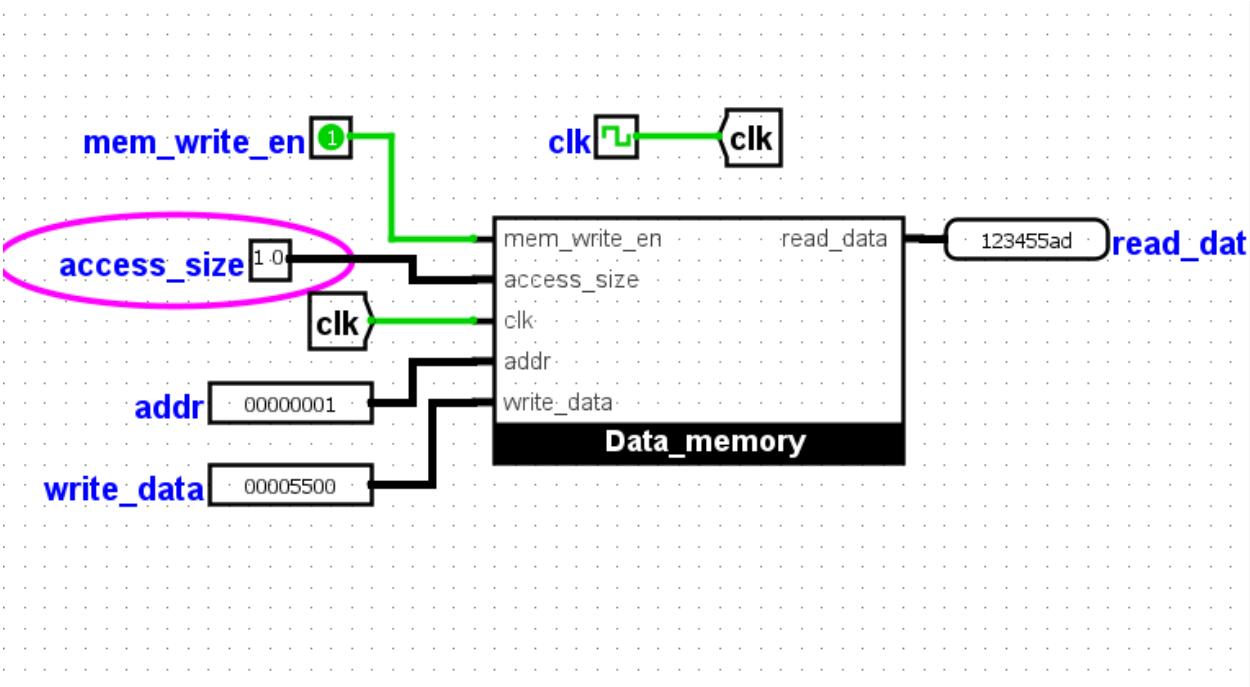
Test Case 7: Positive Load and Byte Masking

I changed the access size to Byte at address 3. The output was 0x00000012. Because 12 is positive, the circuit padded the top with zeros. I also verified that writing a single byte does not change the other bytes in the same word.



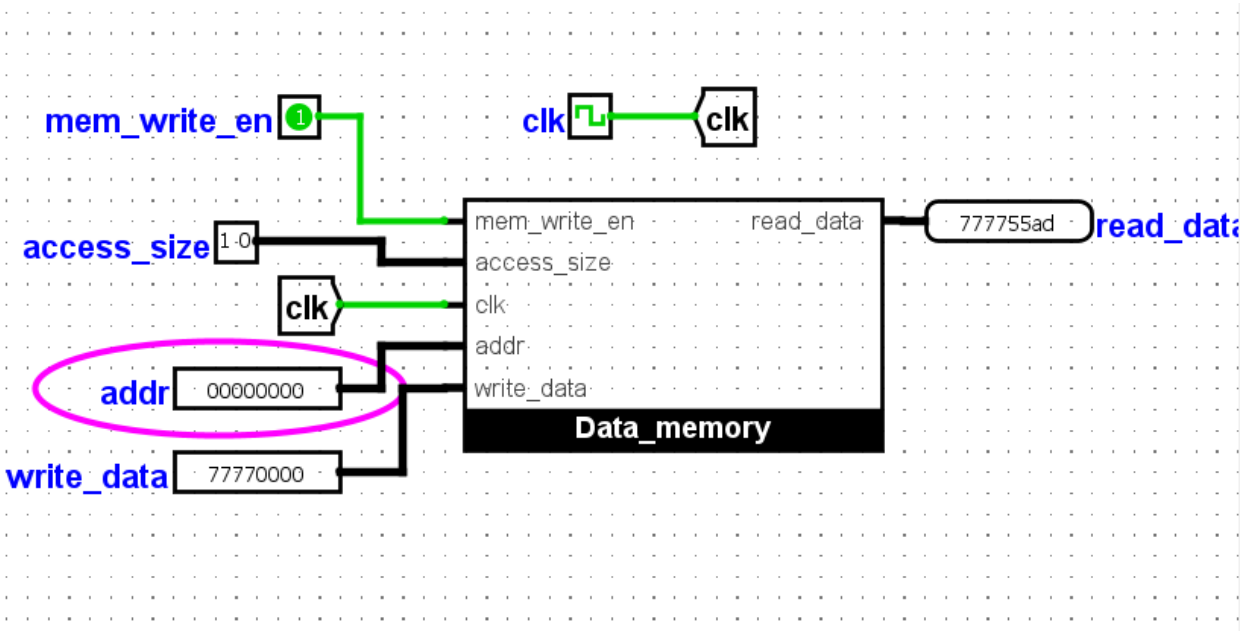
Test Case 8: Byte Masking (SB Write)

I set addr to 1, access_size to 00, and write_data to 0x00005500. After a clock pulse, I read back the full word. The result 0x123455AD proved only Bank 1 was updated while the other bytes remained unchanged.



Test Case 9: Halfword Masking (SH Write)

I set addr to **2**, access_size to **01** (Halfword), and write_data to **0x77770000**. After pulsing the clock, I read the full word at address 0. The result **0x777755AD** proves that the upper two banks (Bank 2 and 3) were updated with 7777 while the lower halfword 55AD remained untouched.



Note on Alignment

- All test cases were performed using aligned addresses (multiples of 4 for words and multiples of 2 for halfwords) as required by the lab specifications. Misaligned accesses were considered out of scope for this design.